

Exploring Noise Schedulers in Diffusion Models

Pranav Balakrishnan

pranavbalakr@umass.edu

Sidisha Shyam Barik

sbarik@umass.edu

Abstract

This project aims to develop identify the importance of noise levels on diffusion models. Our primary dataset, CIFAR-10, will help evaluate various noise schedulers including their scaled and shifted counterparts on Variance-Preserving (VP), and Sub-Variance-Preserving (Sub-VP) diffusion formulations, to identify the best approach for a given formulation.

1. Introduction

Diffusion models have gained attention for their high-quality generative capabilities. The primary objective of this project is to understand how different noise schedulers impact the training as well as sampling of diffusion models and how these noise schedulers perform under different diffusion model formulations.

Noise schedulers help define the amount and the way the noise is introduced in the images while training. This directly controls the noise levels that the model focuses on. We can think of it in terms of human eyes and ears which are far more sensitive to low frequencies than high ones. Therefore, allowing diffusion models to prioritize modeling lower frequencies, which align with higher noise levels could be beneficial. On the other hand, focusing on the high frequency parts of the image can lead to less blurry and better textured images. Therefore, finding the optimal balance between noise levels during model training can lead to the generation of better quality images. There are several factors that affect the noise levels along with the noise scheduler including model output parametrization, the loss weighting and the time step distribution [2]. In this work, we focus on the noise level distribution and how they affect training and sampling.

In the diffusion process, we progressively add noise to the data as: $z_t = \alpha(t)x + \sigma(t)\epsilon$ where $\alpha(t)$ is a time dependent scale factor that rescales the original input to stop it from growing uncontrollably and $\sigma(t)$ referred to as the noise schedule, indicates the total amount of noise added at timestep t .

We use the cosine noise schedule in the Variance-

Preserving (VP) formulation introduced by Nichol & Dhariwal (2021) [12] as our baseline and compare it with Sine based noise schedule in the Sub-Variance-Preserving (Sub-VP) formulation[11] and Laplace Noise Schedulers in both Variance-Preserving (VP), and Sub-Variance-Preserving (Sub-VP) formulations. We also try scaling these distributions in different ways to understand how aggressively focusing on a particular noise level (Mid Range) affects the image quality.

We also investigate how using samplers that emphasize different noise levels during the sampling process affects models trained more extensively on specific noise levels. Sampling from a trained diffusion model requires deciding how to distribute the progression through noise levels, from high to low. In regions where certain noise levels are more critical, it is beneficial to spend more time evaluating the model, which involves spacing those levels more closely. However, since the number of sampling steps is typically constrained, this necessitates wider spacing in less critical regions. Another key consideration is the potential accumulation of errors: mistakes made at high noise levels early in the process can propagate and cause greater issues than those made later at low noise levels. As a result, the optimal balance of noise levels during training and sampling can differ significantly.

The main evaluation metric that we are using is the FID Score. Calculation the FID score is a very time and compute intensive endeavour. So, in order to quickly iterate and refine our models we use a custom score for CIFAR-10 dataset that we discuss further in Section 3.

2. Related work

The initial Denoising Diffusion Probabilistic Models (DDPM) paper [5] introduced the linear scheduler which was superseded by the cosine noise schedule introduced by Nichol & Dhariwal (2021) [12]. They showed that this works better as linearly increasing variance often results in excessive noise, especially in high-resolution images. Following this, Karras et al. (2022) [8] proposed a simplified variance-exploding (VE) noise schedule $\sigma(t) = t$, wherein they eliminated the the time variable entirely, directly con-

trolling noise with the noise schedule instead. More recent works by Chen (2023) [1] and Hoogetboom et al. (2023) [6] emphasize adjusting noise schedules according to the signal-to-noise ratio (SNR) to address higher correlations in adjacent pixel values of high-resolution images.

SD3 Esser et al. (2024) [3] empirically found that giving higher weights to intermediate noise intensities is more effective during training. Adding to this, in their recent work, Hang et al. (2024) [4] showed that the importance sampling of the logarithm of the Signal-to-Noise ratio (log SNR) is particularly beneficial for training efficiency when increasing the sample frequency around $\lambda = \log \text{SNR} = 0$. Their work also experiments with various noise schedules like Laplace, Cauchy, and Cosine Shifted/Scaled revealing that the Laplace schedule performs best under the VP formulation.

Our work builds on these approaches by comparing the baseline Cosine Schedule with Sine and Laplace noise schedules across both VP and Sub-VP formulations to identify the most effective approach for different scenarios. We further investigate the impact of scaling noise levels and test various schedules and formulations during sampling to determine whether Bayesian sampling yields optimal results or if alternative methods such as Cosine, EDM, Laplace, or shifted Cosine Schedulers perform better.

3. Method

We focus on a basic DDPM model.

The forward process : We assume that our images $x \sim p_{\text{data}}(x)$. We will denote the series of latent variables z_t for timesteps $t \in [0, 1]$: $z_0, \dots, z_1$. We get these z_t by increasingly noisifying the initial image x . We can denote a particular z_t as:

$$z_t = \alpha_t x + \sigma_t \epsilon \quad (1)$$

We will be formulating all our computations in terms of the logSNR ($\lambda = \alpha(t)^2/\sigma(t)^2$) because it directly tells us the amount of information obscured by noise. This makes it possible for us to compare different models, different noise schedules, and different formulations. Another reason for doing so is though time itself is linear, the total noise added $\sigma(t)^2$ is not a linear function of t . Therefore the noise is not being added in a linear way. Therefore, we will be using the unified formulation as introduced by Kingma et al.[9] to represent our z_t as:

$$x \sim p_{\text{data}}(x)$$

$$z_t = \alpha_\lambda x + \sigma_\lambda \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (2)$$

$$\lambda = \log \text{SNR}(t), \quad \text{SNR}(t) = \frac{\alpha_t^2}{\sigma_t^2} = \frac{\alpha_\lambda^2}{\sigma_\lambda^2} \quad (3)$$

This works as λ is strictly monotonically decreasing in time t . Therefore as we move further, the signal-to-noise ratio decreases indicating a growing noise level in the image (z_t).

We can write the distribution of the latent variables z_t conditioned on x as:

$$q(z_t|x) = \mathcal{N}(\alpha_{\lambda_t} x, \sigma_{\lambda_t}^2 I) \quad (4)$$

We can generalize this by saying that while moving from timestep s to timestep t where $0 \leq s < t \leq 1$, the distribution is given by:

$$q(z_t|z_s) = \mathcal{N}(\alpha_{\lambda_t|\lambda_s} z_s, \sigma_{\lambda_t|\lambda_s}^2 I) \quad (5)$$

where

$$\alpha_{\lambda_t|\lambda_s} = \frac{\alpha_{\lambda_t}}{\alpha_{\lambda_s}}, \quad (6)$$

and

$$\sigma_{\lambda_t|\lambda_s}^2 = \sigma_{\lambda_t}^2 - \alpha_{\lambda_t|\lambda_s}^2 \sigma_{\lambda_s}^2 \quad (7)$$

We use this to incrementally noisify the image. Our model then learns the noise that was added to the image while noisifying it to a latent state z_t from x .

We used the following loss as our objective as mentioned by Kingma et al. [9]:

$$\mathcal{L}_w(x) = \frac{1}{2} \mathbb{E}_{\lambda \sim p(\lambda), \epsilon \sim \mathcal{N}(0, I)} \left[\frac{w(\lambda)}{p(\lambda)} \cdot \|\hat{\epsilon}_\theta(z_\lambda; \lambda) - \epsilon\|^2 \right] \quad (8)$$

Here, we see that the loss itself is weighted based on the noise levels (i.e. for each sampled t , when the model computes the corresponding λ , the model weighs the squared loss by a factor of $w(\lambda_t)/p(\lambda_t)$. This introduces an additional complexity of how the noise levels impact the model training. In order to remove this additional dependence, we set our $w(\lambda_t)$ in such a way that it equals $p(\lambda_t)$ thereby weighting the squared error by 1 and our objective simply becomes:

$$\mathcal{L}_w(x) = \frac{1}{2} \mathbb{E}_{\lambda \sim p(\lambda), \epsilon \sim \mathcal{N}(0, I)} \left[\|\hat{\epsilon}_\theta(z_t; \lambda_t) - \epsilon\|^2 \right] \quad (9)$$

We are exploring 2 formulations, the VP formulation as well as the sub-VP formulation. In VP Diffusion, we set $\alpha(t) = \sqrt{1 - \sigma(t)^2}$ and also require $\sigma(t) \leq 1$. In this case, if $\text{Var}[\mathbf{x}_0] = 1$ then it can be shown that $\text{Var}[\mathbf{x}_t] = 1$ for all t . Therefore, the total variance is 1 at every step. In Sub VP Diffusion, we set $\alpha(t) = 1 - \sigma(t)$. This leads to the $\text{Var}[\mathbf{x}_t] \leq 1$ if $\text{Var}[\mathbf{x}_0] = 1$. This can result in a reduction of the number of sampling steps needed for good quality images because straighter paths through input space between data and noise will be followed. This can be seen from the behavior observed in the Figure 1.

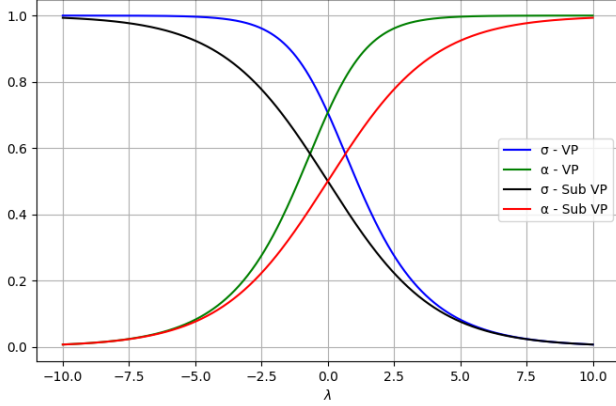


Figure 1. Standard Deviation (σ) and Scaling Factor(α) observed in case of VP and Sub-VP formulation w.r.t λ

More concretely, in VP, we set:

$$\alpha_\lambda = \sqrt{1 - \sigma_\lambda^2} \implies \alpha_\lambda^2 = 1 - \sigma_\lambda^2 \quad (10)$$

$$\lambda = \log \frac{\alpha_\lambda^2}{\sigma_\lambda^2} = \log \frac{1 - \sigma_\lambda^2}{\sigma_\lambda^2}$$

$$e^\lambda = \frac{1 - \sigma_\lambda^2}{\sigma_\lambda^2} \implies \sigma_\lambda^2 = \frac{1}{1 + e^\lambda} \quad (11)$$

Therefore, our noisify step in forward process looks like:

$$z_t = \sqrt{1 - \sigma_\lambda^2} x + \sigma_\lambda \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (12)$$

In Sub-VP, we set:

$$\alpha_\lambda = 1 - \sigma_\lambda \quad (13)$$

$$\lambda = \log \frac{\alpha_\lambda^2}{\sigma_\lambda^2} = \log \frac{(1 - \sigma_\lambda)^2}{\sigma_\lambda^2}$$

$$e^{\lambda/2} = \frac{1}{\sigma_\lambda} - 1 \implies \sigma_\lambda = \frac{1}{1 + e^{\lambda/2}} \quad (14)$$

The above obtained α and σ are plotted in the Figure : 1

Therefore, our noisify step in forward process looks like:

$$z_t = (1 - \sigma_\lambda) x + \sigma_\lambda \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (15)$$

Since, λ ranges from $-\infty$, to $+\infty$ as t ranges from 0 to 1, we limit the values of λ between a λ_{min} and λ_{max} for numerical stability as per the truncation procedure mentioned by Kingma et al. [9]

Note, since $\lambda = f_\lambda(t)$ and $t \sim U(0, 1)$, we can derive $p(\lambda)$ using the change of variable formula:

$$p(\lambda) = p(t) \left| \frac{dt}{d\lambda} \right| \implies p(\lambda) = \left| \frac{dt}{d\lambda} \right| \quad (16)$$

	$p(\lambda)$
Cosine VP	$\text{sech} \left(\frac{s\lambda}{2} \right) \cdot \frac{s}{2\pi}$
Laplace VP and Sub-VP	$\frac{1}{2b} \exp \left(-\frac{ \lambda - \mu }{b} \right)$
Sine Sub-VP	$\frac{1}{\pi} \cdot \left(\frac{e^{\lambda/2}}{\sqrt{e^\lambda + 2e^{\lambda/2}}} \right) \cdot \frac{1}{1 + e^{\lambda/2}}$

Table 1. $p(\lambda)$ for different Noise Schedulers under different formulations before truncation

	$\lambda(t)$
Cosine VP	$\frac{-2}{s} \log \tan \left(\frac{\pi t}{2} \right)$
Laplace VP and Sub-VP	$\mu - b \text{sgn}(0.5 - t) \log(1 - 2 t - 0.5)$
Sine Sub-VP	$2 \log \left((1/\sin \frac{\pi t}{2}) - 1 \right)$

Table 2. $\lambda(t)$ for different Noise Schedulers under different formulations before truncation

since $p(t)$ is uniform.

Since, λ is strictly monotonically decreasing in time:

$$p(\lambda) = -\frac{dt}{d\lambda} \quad (17)$$

We now look at the Noise Schedules that we used under the VP and sub-VP formulation.

1. Cosine Noise Schedule in VP formulation:

Here we set:

$$\sigma_\lambda = \sin \left(\frac{\pi t}{2} \right)$$

$$\alpha_\lambda = \sqrt{1 - \sigma_\lambda^2} = \cos \left(\frac{\pi t}{2} \right)$$

We can therefore derive our $\lambda = f_\lambda(t)$, $t = f_t^{-1}(\lambda)$ and $p(\lambda)$ as:

$$\lambda = f_\lambda(t) = \log \frac{\cos^2 \frac{\pi t}{2}}{\sin^2 \frac{\pi t}{2}} = -2 \log \tan \left(\frac{\pi t}{2} \right)$$

$$f_t^{-1}(\lambda) = t = \frac{2}{\pi} \tan^{-1} \left(e^{-\lambda/2} \right)$$

$$p(\lambda) = -\frac{dt}{d\lambda} = \text{sech} \left(\frac{\lambda}{2} \right) \cdot \frac{1}{2\pi}$$

Thus, we can see that now when we sample our time steps from $t \sim U(0, 1)$, our noise levels are following the distribution $p(\lambda)$. We can observe the distribution of the noise levels under this formulation in this Figure 9.

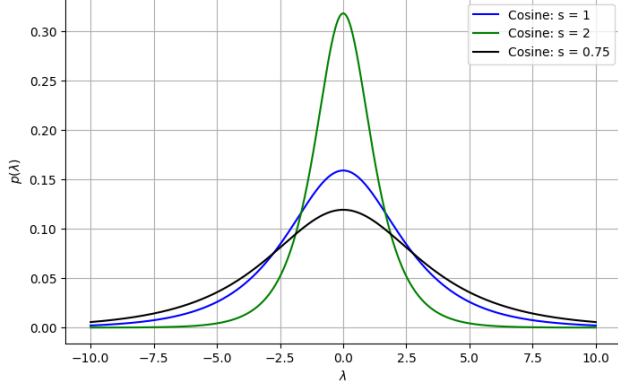


Figure 2. $p(\lambda)$ for Cosine Noise Scheduler under VP formulation before truncation

2. Sine Noise Schedule in sub-VP formulation:

Similar to what we did for the Cosine Noise Schedule in the VP formulation, we start by setting:

$$\sigma_\lambda = \sin\left(\frac{\pi t}{2}\right)$$

$$\alpha_\lambda = 1 - \sin\left(\frac{\pi t}{2}\right)$$

and therefore, we get the following:

$$\lambda = f_\lambda(t) = \log \frac{(1 - \sin \frac{\pi t}{2})^2}{(\sin \frac{\pi t}{2})^2} = 2 \log \left(\frac{1}{\sin \frac{\pi t}{2}} - 1 \right)$$

$$f_t^{-1}(\lambda) = t = \frac{2}{\pi} \sin^{-1} \left(\frac{1}{1 + e^{\lambda/2}} \right)$$

$$p(\lambda) = -\frac{dt}{d\lambda} = \frac{1}{\pi} \cdot \left(\frac{e^{\lambda/2}}{\sqrt{e^\lambda + 2e^{\lambda/2}}} \right) \cdot \frac{1}{1 + e^{\lambda/2}}$$

The name Cosine Noise Schedule comes from the fact that in VP, setting $\alpha_\lambda = \sqrt{1 - \sigma_\lambda^2}$ when $\sigma_\lambda = \sin\left(\frac{\pi t}{2}\right)$ results in a Cosine function. However, this is not the case in Sub-VP as can be seen above, and therefore, we will refer to this noise schedule as Sine Noise Schedule. We can observe the distribution of the noise levels under this formulation in this Figure 3.

3. Laplace in VP:

For Laplace we start with:

$$p(\lambda) = \frac{1}{2b} \exp\left(-\frac{|\lambda - \mu|}{b}\right)$$

Using equation (17) we can see that:

$$\lambda = f_\lambda(t) = \mu - b \operatorname{sgn}(0.5 - t) \log(1 - 2|t - 0.5|) \quad (18)$$

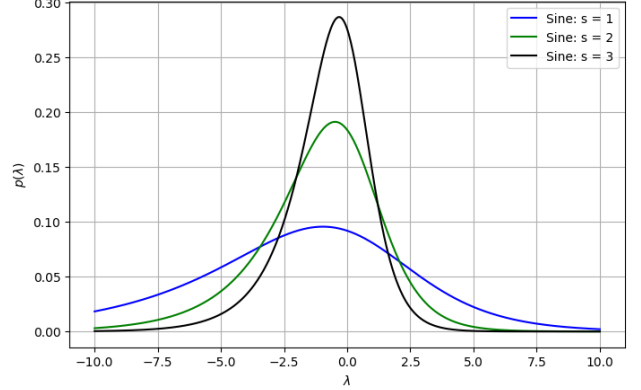


Figure 3. $p(\lambda)$ for Sine Noise Scheduler under Sub-VP formulation before truncation

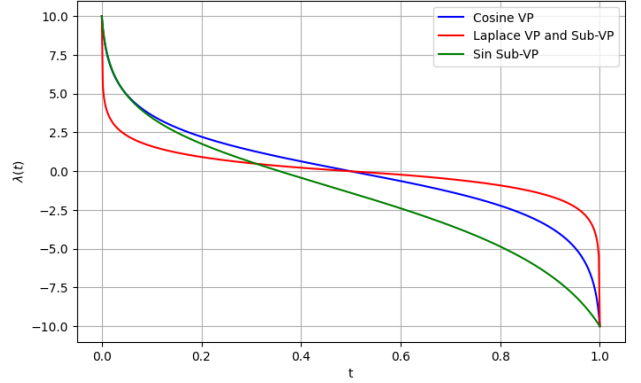


Figure 4. Comparison of how λ behaves in different model formulations after truncation

4. Laplace in Sub-VP:

Since we start with $p(\lambda)$ in the above, both $p(\lambda)$ and $\lambda = f_\lambda(t)$ stay the same in Sub-VP formulation as well. We only update the α and σ as in equation (15). We can observe how the distribution of noise levels in this Figure 7.

For a summarised view of the above, please refer the tables 1 2 and the Figure 5 We decided to choose Cosine/Sine and Laplace samplers for our experimentation as both are centered around $\log \text{SNR}=0$ and we assess weather it is more beneficial to aggressively focus on the central noise levels like Laplace or more broadly as is the case in Cosine/Sine. Also, note that the Sine Noise Schedule's peak is not at $\lambda = 0$ as can be seen in the Figure 3. So, this means that more focus is given to higher noise levels during training. We further explore the scaled versions of all the noise schedulers as well to understand how different scaling factors affect the training.

Sampling:

For sampling, we use the Markov property of the for-

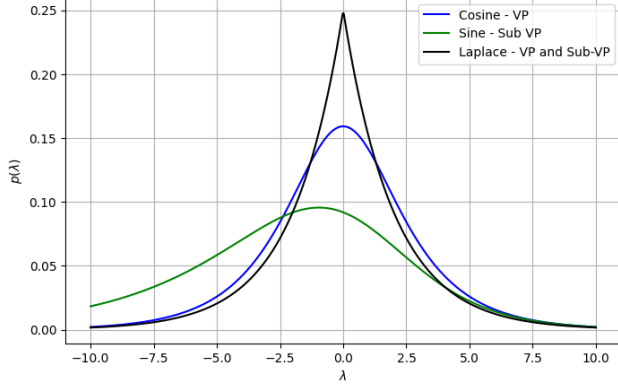


Figure 5. Comparison between the probability density functions of λ , $p(\lambda)$ in different model formulations before truncation

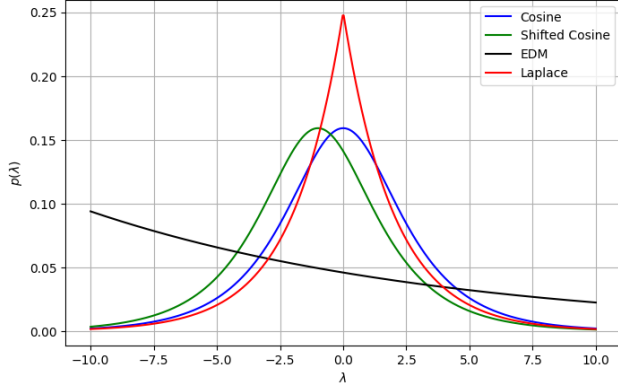


Figure 6. $p(\lambda)$ for different Samplers before truncation

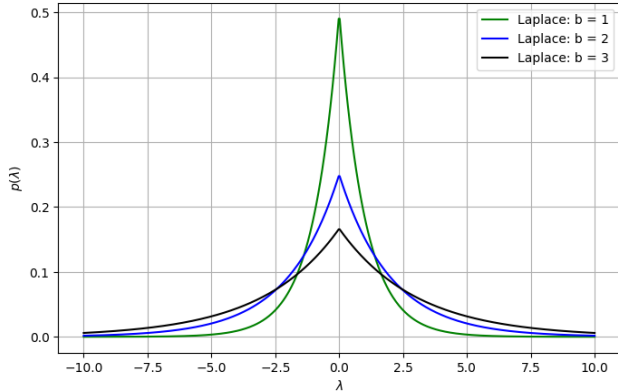


Figure 7. $p(\lambda)$ for laplace Noise Scheduler under VP and Sub-VP formulation before truncation

ward process. Referencing the Kingma et al.[10], we can see that if we have the forward process as mentioned above : $q(z_{\lambda_t}|z_{\lambda_s})$, we can compute the $q(z_{\lambda_s}|z_{\lambda_t}, x)$ as the

	$p(\lambda)$
Cosine	$\text{sech}\left(\frac{s\lambda}{2}\right) \cdot \frac{s}{2\pi}$
Laplace	$\frac{1}{2b} \exp\left(-\frac{ \lambda-\mu }{b}\right)$
Cosine Shifted	$\text{sech}\left(\frac{s(\lambda-\mu)}{2}\right) \cdot \frac{s}{2\pi}$
EDM	$\frac{e^{-\lambda/(2\rho)}}{2\rho(\sigma_{\max}^{1/\rho} - \sigma_{\min}^{1/\rho})}$

Table 3. $p(\lambda)$ for different Samplers before truncation

	$\lambda(t)$
Cosine	$\frac{-2}{s} \log \tan\left(\frac{\pi t}{2}\right)$
Laplace	$\mu - b \text{sgn}(0.5 - t) \log(1 - 2 t - 0.5)$
Cosine Shifted	$\mu + \frac{-2}{s} \log \tan\left(\frac{\pi t}{2}\right)$
EDM	$-2\rho \log\left(\sigma_{\max}^{1/\rho} + (1-t)\left(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}\right)\right)$

Table 4. $\lambda(t)$ for different Samplers

Bayesian posterior:

$$\sigma_Q^2(\lambda_s, \lambda_t) = \sigma_{\lambda_t|\lambda_s}^2 \frac{\sigma_s^2}{\sigma_t^2} \quad (19)$$

$$\mu_Q(z_t, x; \lambda_s, \lambda_t) = \alpha_{\lambda_t|\lambda_s} \frac{\sigma_{\lambda_s}^2}{\sigma_{\lambda_t}^2} z_t + \alpha_{\lambda_s} \frac{\sigma_{\lambda_t|\lambda_s}^2}{\sigma_{\lambda_t}^2} x \quad (20)$$

Since, we do not have the true x , we replace the x by using the output of the denoising model:

$$p(z_s|z_t) = \mathcal{N}(z_s; \mu_\theta(z_t; \lambda_s, \lambda_t), \sigma_Q^2(\lambda_s, \lambda_t)I) \quad (21)$$

where the variance $\sigma_Q^2(\lambda_s, \lambda_t)$ same as given above and

$$\mu_\theta(z_t; \lambda_s, \lambda_t) = \frac{1}{\alpha_{\lambda_t|\lambda_s}} z_t - \frac{\sigma_{\lambda_t|\lambda_s}^2}{\alpha_{\lambda_t|\lambda_s} \sigma_t} \hat{\epsilon}_\theta(z_t; \lambda_t) \quad (22)$$

We modify the α and σ according to the model formulation it was trained on.

In order to assess how different samplers work on our models, we tested out 4 samplers: Cosine, Laplace, Left Shifted Cosine and EDM sampler [8] as defined in tables 3 and 4

We have rewritten the DDPM sampler in the unified formulation as:

1. Define $ts \leftarrow \text{linspace}\left(1 - \frac{1}{\text{steps}}, 0, \text{steps}\right)$
2. Initialize $z_t \sim \mathcal{N}(0, 1)$
3. Initialize preds
4. For each timestep t in (ts):
 - a. Compute $\lambda_t \leftarrow f_\lambda(t)$
 - b. Compute $\lambda_s \leftarrow f_\lambda\left(t - \frac{1}{\text{steps}}\right)$
 - c. Predict noise $\hat{\epsilon} \leftarrow \text{model}(z_t, \lambda_t)$
 - d. Update $z_t \leftarrow \mu_Q(z_t, \hat{\epsilon}, \lambda_t, \lambda_s)$
 - e. If $t \geq \frac{1}{\text{steps}}$, add noise:
$$z_t \leftarrow z_t + \sqrt{\sigma_Q^2(\lambda_t, \lambda_s)} \cdot \text{noise}$$
5. Return z_t

Model Architecture: We conduct all our experiments on a DDPM model trained from scratch with a standard UNet model backbone of medium model size with around 100M parameters. We train the model with a batch size of 256 for 130 epochs using the AdamW optimizer with OneCycleLR and Mixed Precision.

Evaluation Metrics: We use two evaluation metrics (to measure the image quality) - The traditional FID from torchmetrics. Along with this we also used a custom image quality measure (as suggested by Jeremy Howard in his Deep Learning course [7]) specifically tuned for CIFAR-10's image characteristics. We first trained a classifier on the CIFAR-10 dataset that has an 85% accuracy. We then used that classifier to calculate the Frechet distance instead of using the InceptionNet as the backbone. Since the InceptionNet was trained on images on 299x299 images, our CIFAR-10 dataset as well as our samples will be resized from 32x32 to 299x299 while computing the traditional FID metric which could potentially lead to some unreliable FID scores. Another reason for using this custom score is that our CIFAR-10 classifier is much smaller than the InceptionNet and this will allow us to quickly calculate a score thereby helping us iterate and experiment faster since calculating the FID score each time is time consuming and compute expensive. We report our results in the traditional FID and perform our iterative experiments using our custom score. We generate 3000 images using each model that we trained using the EDM sampler[8] and then compute the FID scores.

4. Results

We compared the 4 samplers on our baseline Cosine VP model and we observed that the Cosine and EDM samplers

Sampler	FID
Cosine	167.8155
Cosine Shifted	174.0111
Laplace	168.1270
EDM	167.8568

Table 5. FID-3k scores observed for different samplers using samples generated using baseline Cosine Noise Scheduler model in VP formulation

Model	FID
Cosine (s = 0.75)	168.7814
Cosine (s = 1)	167.8568
Cosine (s = 2)	173.1677
Laplace (b = 1)	173.6640
Laplace (b = 2)	168.9630
Laplace (b = 3)	168.2587

Table 6. FID-3K Score Comparison for models trained in VP Formulation

Model	FID
Sine (Scale s = 1)	168.7536
Sine (Scale s = 2)	173.6887
Sine (Scale s = 3)	183.6775
Laplace (b = 1)	174.2637
Laplace (b = 2)	169.6126
Laplace (b = 3)	168.9708

Table 7. FID-3K Score Comparison for models trained in Sub-VP Formulation

were the best performing as can be seen from this table 5. Based on the FID-3k scores and eye test, we sampled all of the remaining models using the EDM sampler. Since, mistakes made at high noise levels early in the process can propagate and cause greater issues than those made later at low noise levels we expected the EDM sampler which is biased towards the higher noise levels to perform better. However, since the comparison was performed using FID scores of 3000 generated samples, we were unable to observe and significant difference between the Cosine and EDM samplers. Additional model training and sample generation is needed to make concrete conclusions.

The FID values shown in the tables 6 and 7 were calculated using only 3000 generated images for each model due to compute limitations. Further, all the models were only trained for 25k iterations. From the table we see that the best performing models are : baseline Cosine model (VP formulation) and Laplace (b = 3) model (VP formulation). According to the recent work by Hang et al. (2024) [4], the Laplace Scheduler which more aggressively focuses



Figure 8. Cosine ($s = 0.75$) VP



Figure 9. Cosine ($s = 1$) VP



Figure 10. Cosine ($s = 2$) VP



Figure 11. Laplace ($b=1$) VP



Figure 12. Laplace ($b = 2$) VP



Figure 13. Laplace ($b = 3$) VP

on a particular noise level outperforms the baseline Cosine model under VP formulation. However, we generally observe that aggressively focusing too much on the mid range noise levels is not very helpful in both VP and Sub-VP formulations. So, we need to investigate this further since we are drawing conclusions about models that have very similar FID scores. Similar works calculate the FID with 50k generated samples and train models for millions of iterations and therefore, in order to draw more concrete conclusions, we would need to train our models for more iterations and compute the FID scores using more generated samples. Few of our generated samples from each model can be seen above.



Figure 14. Sine ($s = 1$) Sub VP



Figure 15. Sine ($s = 2$) Sub VP



Figure 16. Sine ($s = 3$) Sub VP



Figure 17. Laplace ($b = 1$) Sub VP



Figure 18. Laplace ($b = 2$) Sub VP



Figure 19. Laplace ($b = 3$) Sub VP

5. Conclusion

In this study, we compared the performance of various samplers on a baseline Cosine VP model, observing that the Cosine and EDM samplers yielded the best results in terms of FID-3k scores and visual quality. While the EDM sampler’s bias toward higher noise levels was hypothesized to offer an advantage due to error propagation considerations, no significant difference was observed between the Cosine and EDM samplers under the given experimental setup.

Future Work:

1. Expanded Training: Extend model training to millions of iterations to better capture long-term performance trends and avoid premature conclusions.
2. Increased Sampling: Calculate FID scores using

- 50k generated samples for CIFAR-10, aligning with industry-standard practices to enhance result reliability.
3. Enhanced Evaluation: Explore additional evaluation metrics beyond FID to capture nuanced differences between models.
 4. Robust Analysis: Investigate the impact of noise scheduling on VP and Sub-VP formulations more rigorously, particularly the trade-offs associated with focusing on specific noise levels (Shifted Left and Shifted Right).
 5. Broader Comparisons: Include additional noise schedulers and samplers to identify configurations that consistently outperform under different formulations.
 6. Computational Optimization: Address compute limitations to enable more extensive experimentation, facilitating deeper insights.

References

- [1] Ting Chen. On the importance of noise scheduling for diffusion models. In *arXiv:2301.10972*, 2023. [2](#)
- [2] Sander Dieleman. Noise schedules considered harmful, 2024. [1](#)
- [3] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *arXiv:2403.03206*, 2024. [2](#)
- [4] Tiankai Hang and Shuyang Gu. Improved noise schedule for diffusion training. In *arXiv:2407.03297*, 2024. [2](#), [6](#)
- [5] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020. [1](#)
- [6] Emiel Hooeboom, Jonathan Heek, and Tim Salimans. Simple diffusion: End-to-end diffusion for high resolution images. In *arXiv:2301.11093*, 2023. [2](#)
- [7] Jeremy Howard. Practical deep learning, 2022. [6](#)
- [8] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *arXiv:2206.00364*, 2022. [1](#), [5](#), [6](#)
- [9] Diederik P. Kingma and Ruiqi Gao. Understanding diffusion objectives as the elbo with simple data augmentation, 2023. [2](#), [3](#)
- [10] Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models, 2023. [5](#)
- [11] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. [1](#)
- [12] A. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021. [1](#)